



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 微处理器原理与应用

实验名称: 实验 2.2

分支程序实验和循环程序实验

专 业 班 级 通信工程 二班

学 生 学 号 202400120045

学 生 姓 名 刘哲旭

实 验 时 间 2025 年 3 月 18 日

实验报告

【实验目的】

- 1. 学习汇编中的控制转移指令及分支结构（包括单分支和双分支）
- 2. 掌握常见循环结构的汇编实现

【实验要求】

- 1. 画好流程图完成实验要求的分支程序实验和循环程序实验

【实验具体内容】

【第一个实验】

(1) 实验流程图

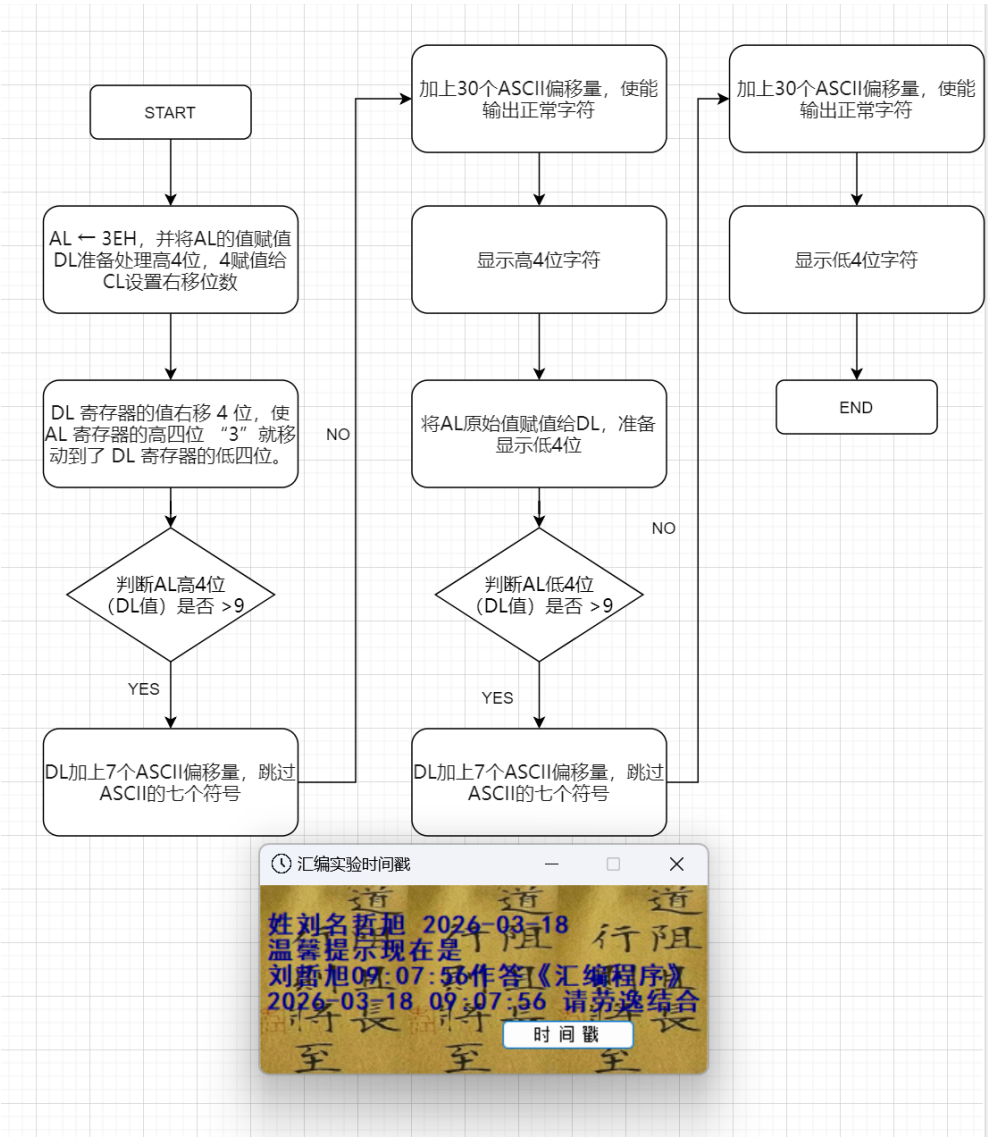


图 1-1 实验 2.2 分支程序实验流程图

(2) 实验源代码（粘贴源代码）：

例如：（第一次实验的源代码）

```
CODE SEGMENT                                ; 代表着代码段的开始
    ASSUME CS:CODE                          ; 告诉汇编器：CS 段寄存器指向我们当前
定义的名为 CODE 的段
START:                                       ; 程序的真正入口点，DOS 加载程序后会跳
转到这里开始执行
    MOV  AL, 3EH                            ; 把十六进制数 3EH（二进制 0011 1110）
装入 AL，这是我们要显示的目标数值
    MOV  BL, AL                            ; 把 AL 的原始值完整备份到 BL，后续
DL 会被多次修改，但原始值还要用来取低四位
    MOV  DL, AL                            ; 再次把 AL 的值复制到 DL，准备先处理
高 4 位
    MOV  CL, 4                             ; 把右移次数 4 放入 CL，供后面的 SHR
指令使用
    SHR  DL, CL                            ; 把 DL 逻辑右移 4 位，高 4 位被移到低
4 位位置（3EH → 03H）
    CMP  DL, 9                             ; 比较 DL 和 9，判断这个 4bit 值是 0~9
还是 A~F（10~15）
    JBE  NEXT1                             ; 如果 DL ≤ 9（无符号比较），直接跳到
加 '0' 的步骤
    ADD  DL, 7                             ; 如果大于 9，先加上 7，为转成字母 A~F
做偏移准备（后面还会再加 30H）
NEXT1:
    ADD  DL, 30H                           ; 转换成 ASCII 字符：0~9 变成 '0'~'9'，
10~15 变成 'A'~'F'（这里应该是 '3'）
    MOV  AH, 2                             ; 设置 DOS 功能号 02H → 显示单个字符
    INT  21H                               ; 调用 DOS 中断 21H，把 DL 中的字符
('3') 显示到屏幕
    MOV  DL, BL                            ; 把一开始保存的原始数值（3EH）重新放
回 DL，准备处理低 4 位
    AND  DL, 0FH                           ; 与 00001111 做与运算，只保留低 4 位，
高 4 位清零（得到 0EH）
    CMP  DL, 9                             ; 再次比较 DL 和 9，判断低 4 位是数字还
```

是字母

```
JBE NEXT2 ; 如果 ≤ 9, 直接跳到加 30H 转字符
ADD DL, 7 ; 大于 9, 先加 7, 为转成 A~F 预留偏移量
NEXT2:
ADD DL, 30H ; 转换成对应的 ASCII 字符(0EH + 7 + 30H
= 45H → 'E')
MOV AH, 2 ; 再次设置功能号 02H, 准备显示下一个字
符
INT 21H ; 调用中断, 把低 4 位对应的字符('E')显
示出来, 此时屏幕应显示 “3E”
MOV AH, 4CH ; 设置 AH = 4CH, 表示 “终止程序并返回
DOS”
INT 21H ; 调用 DOS 中断 21H, 程序正常结束, 控
制权交回 DOS
CODE ENDS
END START ; 告诉汇编器: 程序的入口地址是 START
标号
```

(3) 实验过程

①将代码的文本文件命名为 `branch1.asm`, 保存在 `masm5` 文件夹中, 打开 DOSBOX 自
动挂载到 `masm5` 文件夹, 输入指令 `masm branch1` 并按回车三次, 如图 1-2 所示

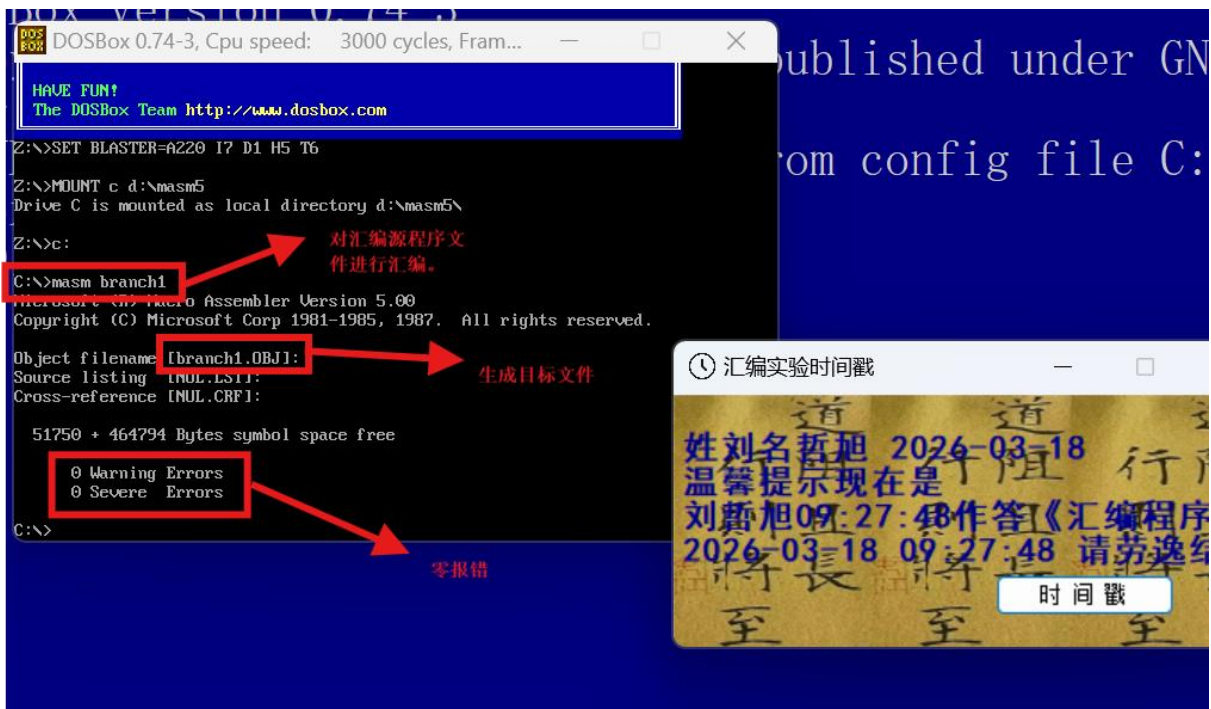


图 1-2 编译生成目标文件

②输入命令 link branch1 并回车三次，生成可执行文件，如图 1-3 所示

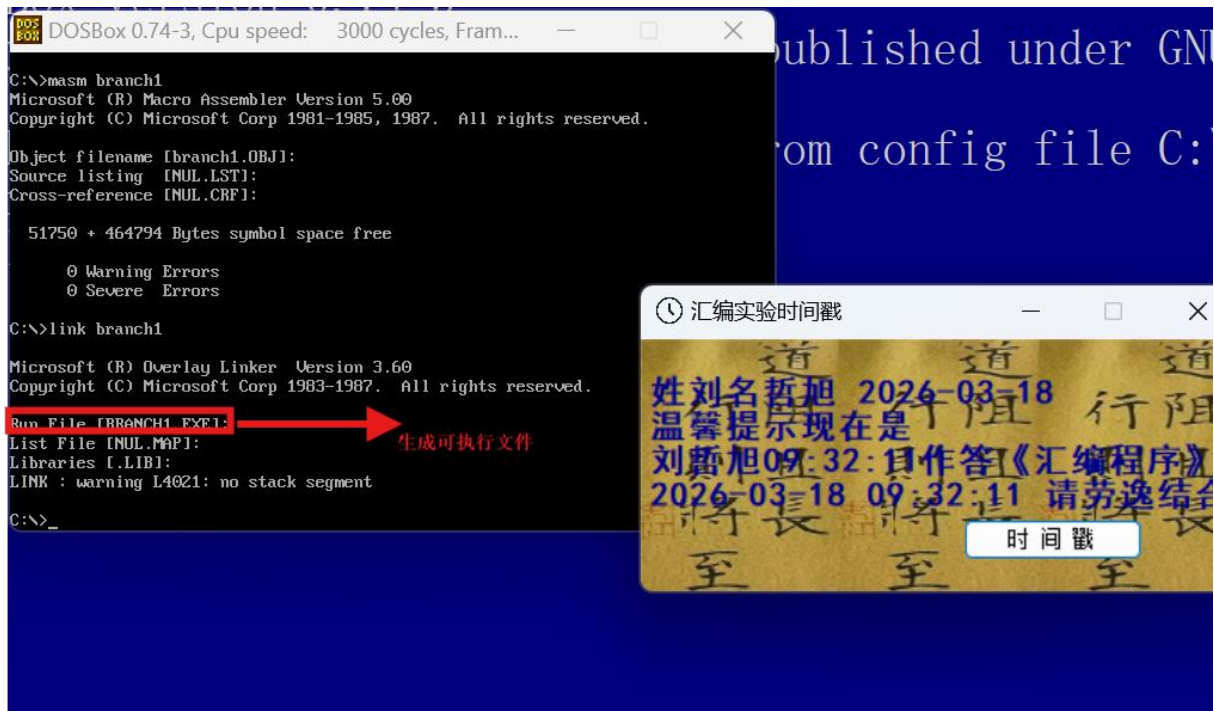


图 1-3 生成 branch1.exe 文件

③输入 branch1，执行 branch1.exe，显示 3E,r 如图 1-4 所示

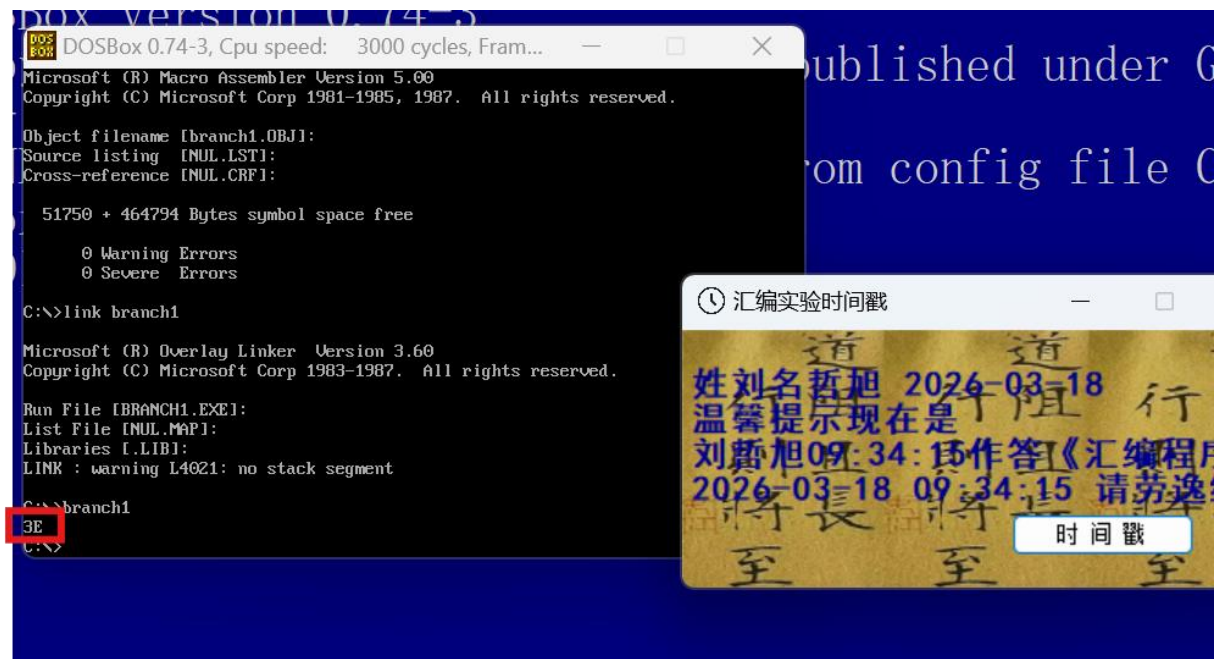


图 1-4 执行 branch.exe 结果显示 3E

【第二个实验】（时间戳）

(1) 实验流程图：

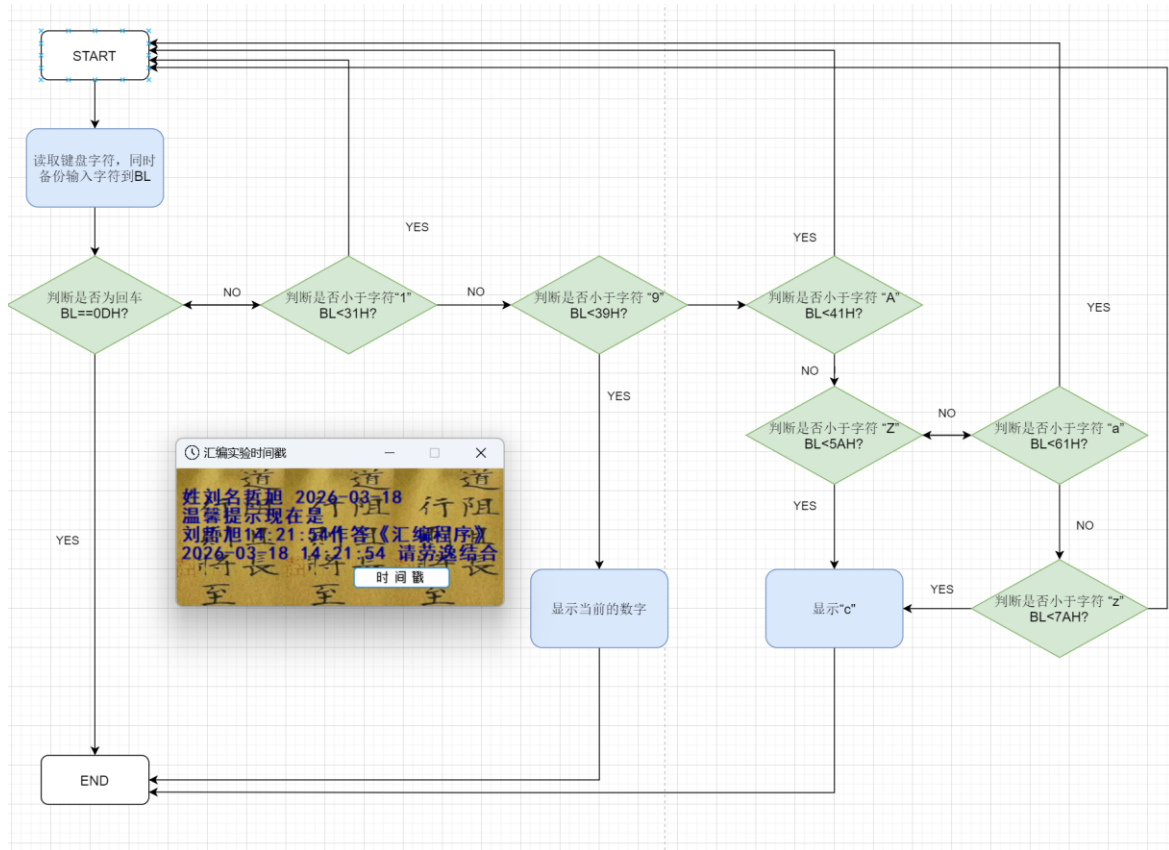


图 2-1 实验二流程图

(2) 实验源代码（粘贴源代码）：

<pre> ASSUME CS:CODE CODE SEGMENT START: MOV AH, 01H INT 21H MOV BL, AL CMP BL, 0DH JE s CMP BL, 31H JB START </pre>	<pre> ; CS 指向名为 CODE 的段 ; 16 位实模式程序必须写此句确保段寻址正确 ; 定义代码段开始 ; 程序入口标号 ; DOS 加载后 CS:IP 指向这里 ; AH=01H: DOS 功能, 从键盘读字符 ; 调用中断 21H, 输入字符 ASCII 码存入 AL ; 同时字符会自动显示在屏幕 (回显) ; 把 AL (输入字符) 备份到 BL ; 防止后续 CMP 破坏 AL, 用于多次比较 ; 比较 BL 是否为回车 (0DH = Enter) ; 是回车 -> 跳转结束程序 ; 比较 BL 是否 < '1' (31H) ; < '1' -> 跳回重新读, 不输出 </pre>
--	--

CMP BL, 39H	; 比较 BL 是否 <= '9' (39H)
JBE OUTPUT1	; 是 1~9 → 去显示原输入字符
CMP BL, 41H	; 比较 BL 是否 < 'A' (41H)
JB START	; < 'A' → 跳回重新读
CMP BL, 5AH	; 比较 BL 是否 <= 'Z' (5AH)
JBE OUTPUT2	; 是 A~Z 则显示 'c'
CMP BL, 61H	; 比较 BL 是否 < 'a' (61H)
JB START	; < 'a' → 跳回重新读字符
CMP BL, 7AH	; 比较 BL 是否 <= 'z' (7AH)
JBE OUTPUT2	; 如果是 a~z 则显示 'c'
JMP START	; 其他字符 → 无输出, 继续循环
OUTPUT1:	; 处理 1~9: 显示用户输入的原字符
MOV DL, BL	; 把 BL (原字符) 放入 DL (显示用)
MOV AH, 02H	; AH=02H: 显示单个字符 (DL 中内容)
INT 21H	; 执行显示 (如输入 '5' 则显示 '5')
JMP START	; 返回主循环
OUTPUT2:	; 处理 A-Z/a-z: 显示 'c'
MOV DL, 'c'	; DL = 'c' 的 ASCII 码
MOV AH, 02H	; AH=02H: 显示字符
INT 21H	; 显示 'c'
JMP START	; 返回主循环
s:	; 结束标签
MOV AH, 4CH	; AH=4CH: 终止程序, 返回 DOS
INT 21H	; 执行退出
CODE ENDS	; 代码段结束
END START	; 汇编结束, 入口为 START

(3) 实验过程

①将代码文本命名为 branch2.asm 并保存在 masm5 文件夹中，打开 DOSBOX 自动挂载到 masm5 文件夹，输入命令 masm branch2，编译后生成目标文件，如图 2-2 所示

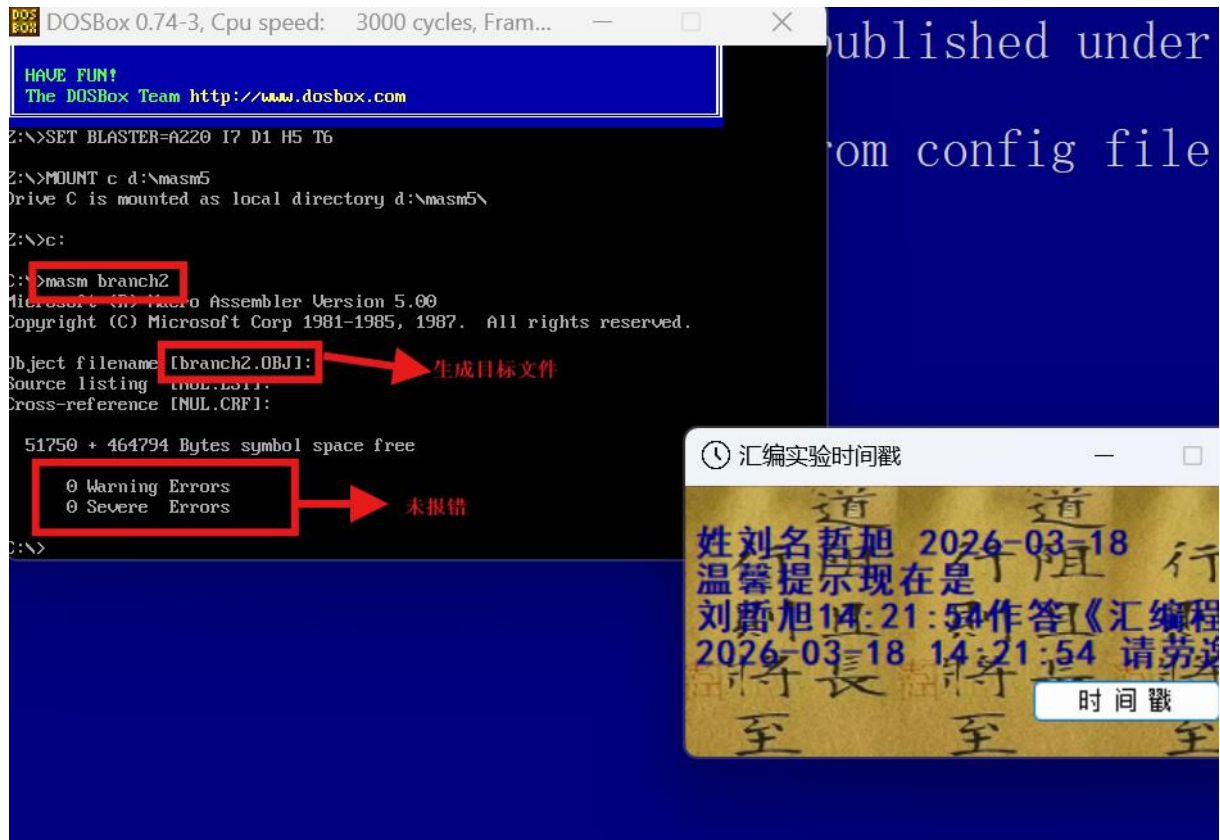


图 2-2 编译生成 branch2.obj 文件

②输入命令 link branch2 并回车三次，生成可执行文件，如图 2-3 所示

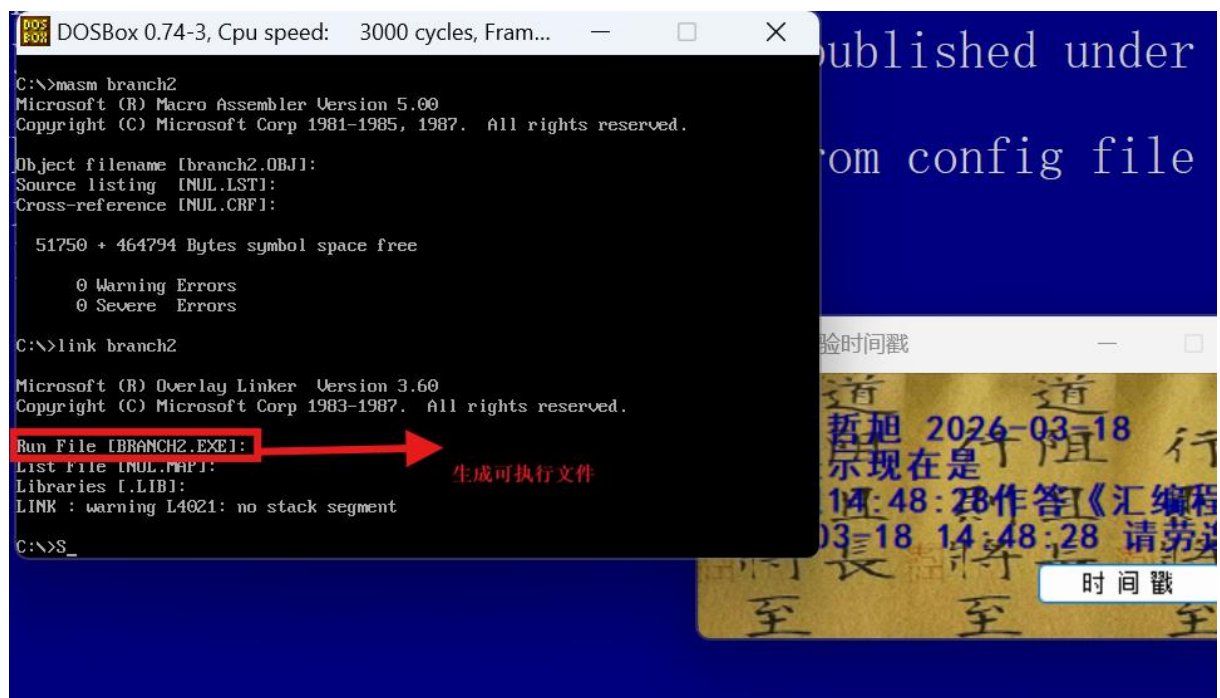


图 2-3 生成 branch2.exe 文件

③输入 branch2 运行 branch2.exe 文件，输入 L 时返回 c，输入 9 时返回 9，输入 4 时返回 4，输入=时不返回，输入回车程序直接结束，满足实验要求，如图 2-4 所示

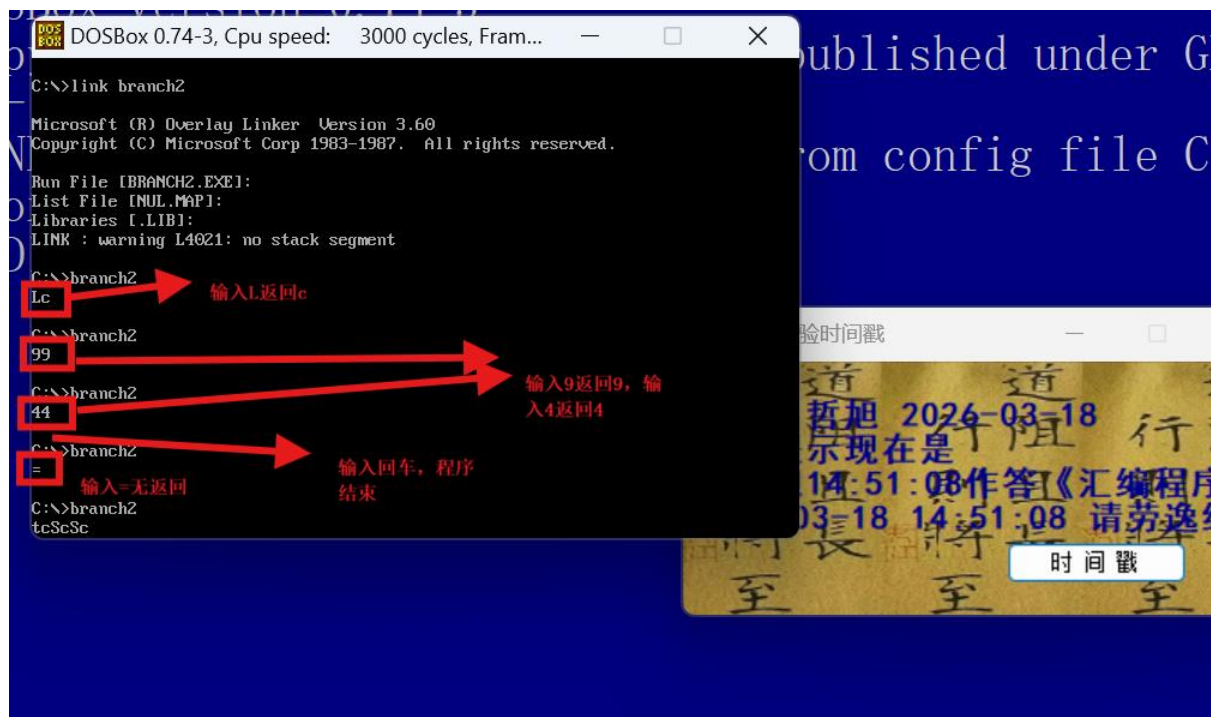


图 2-4 不同输入下的运行结果

【实验心得】

1.对于第一个实验，我在完成代码和注释后，在 DOSBOX 编译后出现报错。下图显示两个报错（error A2106: Line too long 在第 11 行，warning A4001: Extra characters on line 在第 12 行）。经查阅资料得知：MASM 单行最大长度限制通常是 512 字符（包括空格、Tab、注释）。我的代码的第 11 行注释写的字数太多，超过了这个限制。

与此同时，A4001: Extra characters on line MASM 解析某行时，认为指令/标签后面出现了“多余的、非法的字符”。我检查过后并未发现类似于中文输入法下的；这样的异常，但很明显原来的代码的每行注释字数太多，后来我让每行的注释更加精炼，不再出现报错的情况。

```

Copyright (C) Microsoft Corp 1981-1985, 1987. All rights reserved.

Object filename [branch2.OBJ]:
Source listing [NUL.LST]:
Cross-reference [NUL.CRF]:
branch2.ASM(11): error A2106: Line too long
branch2.ASM(12): warning A4001: Extra characters on line

51750 + 448410 Bytes symbol space free

1 Warning Errors
1 Severe Errors

C:\>S
    
```

2. 在这次实验中，我深刻体会到 ASCII 码在汇编程序设计中的核心地位。键盘输入的每一个字符本质上只是一个数值（ASCII 码），而通过简单的比较（CMP）和条件跳转（JB、JBE、JE 等），我们就能把这个数值“翻译”成不同的处理路径——显示原字符、显示固定字符‘c’，或者直接忽略。这种“数值 → 行为”的映射关系，让我联想到一种数学模型——“状态机”，感觉汇编语言虽然底层，却拥有非常直接而强大的控制力。以前总觉得 ASCII 只是字符编码表，现在才真正理解它是如何成为程序逻辑的基石。

3. 通过本次实验，我深刻了解到了写代码之前画流程图的重要性。在第一个实验中，流程图困扰了我许久，我甚至弄错了顺序，直接盯着给出的示范代码画，结果画到一半发现逻辑顺序错了，只好擦掉重来。后来我又复习了一遍汇编指令的相关知识，对实验要求有了一个更加清晰的了解，仔细思考了很久，梳理了各个判断的优先级和跳转目标，再动手画图。对于第二个实验，多个范围判断（1~9、A~Z、a~z）叠加在一起，稍不注意就分不清哪个跳转是针对哪个条件的，好在自己冷静下来正确梳理了逻辑关系，成功画好了流程图。这让我意识到：对于汇编这种低级语言，理解“代码做了什么”比“代码怎么写”更重要，而验证和可视化（流程图）是目前我最有效的理解手段。

【课堂补充】

回答课堂最后的提问部分

1. 四大周期

①时钟周期是处理器运行过程中最基本的时间单位，也称为节拍脉冲周期。它由系统时钟信号决定，其长短与处理器时钟频率成反比。例如，当处理器主频为 1GHz 时，一个时钟周期即为 1 纳秒。时钟周期反映的是 CPU 内部动作推进的最小时间尺度，其他更高层次的周期通常都由若干个时钟周期构成，因此它是理解处理器时序结构的基础。

②总线周期是指 CPU 通过系统总线与主存储器或输入输出设备完成一次信息交换所需要的时间。一次完整的总线周期通常包括发出地址、传送数据以及接收响应等环节，

因此它主要反映的是处理器与外部部件之间的数据通信过程。总线周期的长短不仅与处理器本身的速度有关，还受到总线性能、存储器速度以及外设响应时间等因素的影响。

③机器周期是指 CPU 完成一个基本操作所需要的时间。所谓基本操作，通常包括取指、读存储器、写存储器等底层动作。一个机器周期一般由若干个时钟周期组成，因此其时间长度大于时钟周期。机器周期从功能上描述了 CPU 执行某一基本操作的完整过程，是连接底层时钟节拍与上层指令执行过程的重要中间层次。

④指令周期是指 CPU 执行一条完整指令所需要的全部时间。它通常包括取指令、分析指令、取操作数、执行运算以及保存结果等多个阶段。由于一条指令往往不能通过一次简单操作完成，因此一个指令周期一般由一个或多个机器周期构成。在四类周期之中，指令周期所描述的范围最大，它对应的是程序执行过程中最直接、最完整的操作单位。

从层次关系来看，四大周期之间具有明显的包含关系。通常情况下，时钟周期是最小单位，机器周期由若干时钟周期组成，而指令周期又由若干机器周期组成。总线周期则主要用于描述一次总线访问过程，其侧重点在于 CPU 与外部设备之间的数据传输，因此它与机器周期密切相关，但二者并不完全等同。机器周期强调的是 CPU 完成一个基本操作的时间，而总线周期强调的是一次总线通信所经历的时间过程。

2. 对齐

对齐是指将数据或指令的起始地址安排在某一规定边界上，使其地址满足特定字节数的整数倍要求。对齐的目的在于适应处理器的存储访问规则，提高访存效率，并避免因未对齐访问而产生的性能下降或硬件异常。

在汇编语言中，“对齐”的类别通常可以按对象和按实现方式两个角度进行划分。若用于实验报告，较规范的表述如下：

1. 按对齐对象分类

(1) 数据对齐 (Data Alignment)

指变量、常量或数据段中的各类数据按照规定边界存放。例如，字数据按 2 字节边界对齐，双字数据按 4 字节边界对齐。数据对齐主要用于提高存储访问效率，并满足处理器对数据访问的要求。

(2) 指令对齐或代码对齐 (Instruction / Code Alignment)

指将程序中的指令或代码段起始地址安排在特定边界上。代码对齐有助于处理器更高效地取指和执行，尤其在流水线和高速缓存机制存在时更为重要。

(3) 栈对齐 (Stack Alignment)

指程序运行过程中栈指针保持在规定的地址边界上。例如，在某些体系结构中，函数调用前要求栈按 8 字节或 16 字节对齐。栈对齐通常与函数调用约定、参数传递及局部变量存储有关。

2. 按实现方式分类

(1) 自然对齐 (Natural Alignment)

指数数据按照其自身长度或体系结构默认要求进行对齐。例如，4 字节整数存放在 4 的整数倍地址处，8 字节数据存放在 8 的整数倍地址处。这是最常见、最基本的对齐方式。

(2) 强制对齐或指定对齐 (Forced / Explicit Alignment)

指程序员通过汇编伪指令主动规定对齐边界，使数据或代码从某一特定地址边界开始存放。例如，为了优化性能，可强制某段代码按 16 字节对齐。

(3) 非对齐 (Unaligned)

指数数据或指令的起始地址不满足规定边界要求。非对齐访问在某些平台上会降低执行效率，严重时还可能引发硬件异常。

在汇编语言及系统程序设计中，4K 对齐与奇偶对齐都属于“按地址边界组织数据或代码”的具体形式，只是它们对应的对齐粒度不同、应用场景不同。

1. 4K 对齐

4K 对齐是指将数据或代码的起始地址安排在 4096 字节边界上，即其地址为 4096 的整数倍。该对齐方式常见于页式存储管理和大块数据传输场景，其目的在于满足系统页管理要求，提高数据访问与传输效率。

2. 奇偶对齐

奇偶对齐，通常是指数据按偶地址边界存放，即其起始地址为 2 的整数倍。该方式主要用于保证字数据访问的效率，在某些处理器体系结构中，若数据存放于奇地址，则可能导致额外的访存开销。

关于二者的区别：4K 对齐是指目标对象的起始地址满足 4096 字节整数倍约束，通常用于页式存储管理和大容量数据传输场景；奇偶对齐通常是指数据按偶地址存放，即起始地址满足 2 的整数倍要求，主要用于提高字数据访问效率。两者都属于地址对齐的具体形式，但其对齐粒度和应用场景不同。



知识点

1、控制转移类指令

无条件转移指令 JMP、条件转移指令 JCC

(1) 无条件转移指令：JMP 是汇编语言中的无条件跳转指令。无条件转移指令用于在不进行条件判断的情况下直接改变程序执行地址，是最基本的程序转移方式。

(2) 条件转移指令：条件转移指令通过检测状态标志位的取值来决定是否转移，主要用于实现程序中的判断与分支结构。常见条件转移指令有：

JZ / JE：结果为零或相等时转移

JNZ / JNE：结果不为零或不相等时转移

JC：有进位时转移

JNC：无进位时转移

JS: 结果为负时转移

JNS: 结果非负时转移

JO: 溢出时转移

JNO: 不溢出时转移

JA、JB、JG、JL 等: 用于无符号数或有符号数比较后的转移

其本质是根据标志寄存器中的条件码来决定程序流向。

(3) 循环控制指令:循环控制指令通过计数寄存器和条件标志的共同作用, 实现程序段的重复执行, 是构造循环结构的重要手段。

LOOP: 计数器不为零时循环

LOOPE / LOOPZ: 计数器不为零且零标志满足条件时循环

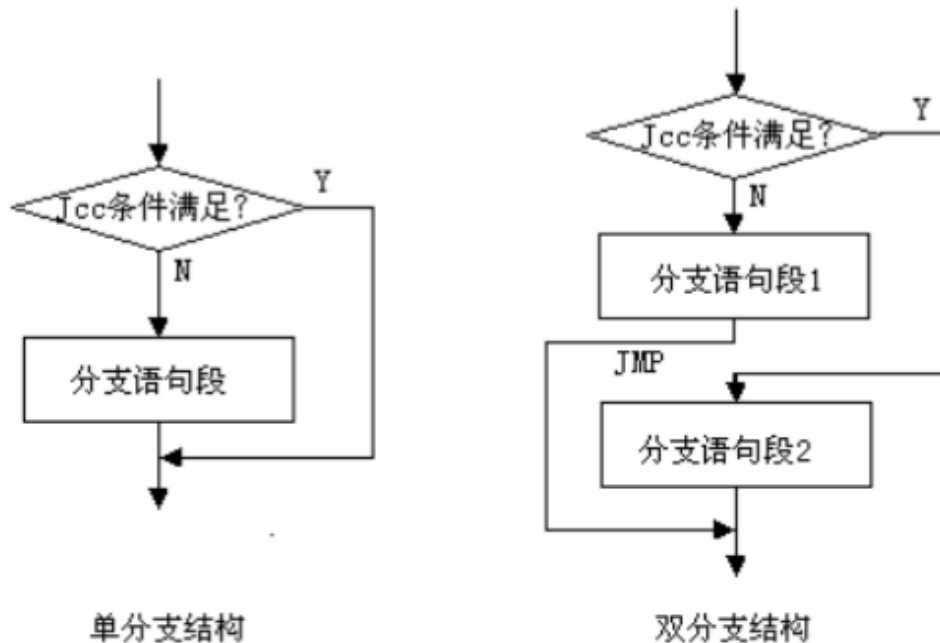
LOOPNE / LOOPNZ: 计数器不为零且零标志不满足条件时循环

这类指令通常与计数寄存器配合使用, 在 x86 中常与 CX 或 ECX 寄存器相关。

3.2 单分支结构与双分支结构

(1) 单分支条件的选择: 当条件满足时, 直接转到分支的下一条指令; 否则, 执行单分支语句段。

(2) 双分支结构中条件的选择任意, 但特别注意: 当分支语句段 1 结束后, 一定要用 JMP 指令跳过分支语句段 2, 即跳到分支结束的位置。



3.3 常用循环结构的汇编语言实现

(1) 对于确定循环次数的循环结构, 一般使用 LOOP 指令构成循环, LOOP 指令中的标号标定循环体的起始位置。但在循环之前要将循环次数送入 CX, 一般还要

与 **PUSH CX** 和 **POP CX** 组合搭配使用。

(2) 对于循环次数不定的循环结构，一般用 **JCC** 指令与标号一起构成循环。

(3) 循环结构的汇编语言程序一般包含三部分，即：

循环初始部分——循环之前准备相应的条件，如循环次数、相关变量/寄存器赋初值等；

循环体——重复执行的程序代码，其中包含改变与循环条件有关的值；

循环控制部分——判断循环条件，决定是否继续循环。